Gradle Deep Dive

For Android · 2026 · From "what is it" to FAANG-level internals

3 build phases | AGP · KSP | Configuration cache | Convention plugins

About this sheet: a single dense reference covering Gradle from first principles through the internals FAANG interviews drill on. Original prose; standard idiomatic Kotlin DSL examples. Pair with the rest of the cheat sheets in this repo.

1. What is Gradle? — core concepts in 60 seconds

Gradle is a build automation tool. Given a project (some source files + dependencies + recipes for what to build), Gradle figures out what tasks need to run, in what order, and runs them — caching everything it can to avoid repeated work.

Four building blocks:

CONCEPT	WHAT IT IS
Project	A unit being built. A typical Android app has multiple — the root project + each module (`:app`, `:feature:home`, etc.). Defined per `build.gradle.kts`.
Task	A single unit of work — compile this, copy that, generate this resource file. Tasks have inputs and outputs and can be skipped if up-to-date.
Plugin	A bundle of tasks + configuration that adds capability. The Android Gradle Plugin (AGP) is what teaches Gradle how to build APKs/AABs.
Build script	A Kotlin (`.kts`) or Groovy (`.gradle`) file that configures the project — applies plugins, declares dependencies, customizes tasks.

Tasks form a **directed acyclic graph (DAG)** — the task graph. Asking Gradle to run `:app:assembleRelease` implicitly schedules every task that produces inputs for it (compile Kotlin, process resources, dex, sign, package…). That's the magic — you only declare relationships; Gradle figures out the order.

2. Why Gradle was created — the build-tool history

ERA	TOOL	WHY IT FAILED FOR MODERN APPS
1976+	Make	Imperative shell-script feel, terrible incremental builds, no dependency management.
2000+	Apache Ant	XML-based, declarative but verbose, no convention; every project reinvented common tasks.
2004+	Apache Maven	Convention over configuration, dependency management built-in. But: rigid lifecycle, hard to customize without writing a plugin in Java, XML is painful for non-trivial logic.
2009+	Gradle	Convention + flexibility. Programmatic build scripts (Groovy/Kotlin DSL). Incremental, parallel, and cacheable by design. Won Android in 2014.
2015+	Bazel (Google)	Hermetic, distributed builds. Used internally at Google. Steeper setup than Gradle; less Android Studio integration.

What would happen without Gradle (or any modern build tool)?

- Every developer would need a wiki page of "how to build" with 30 manual steps
- Dependency versions would conflict; "works on my machine" forever
- Multi-module = manual ordering of compile commands
- No incremental builds — every change re-compiles everything (slow)
- No build caching — same code re-built across machines
- Variants (debug/release/free/paid) = N copies of the project

Gradle solves all of these with one programmable abstraction: the task graph.

3. Gradle vs Maven vs Bazel — comparison

	GRADLE	MAVEN	BAZEL
Build script language	Groovy DSL or Kotlin DSL (.kts)	XML (pom.xml)	Starlark (Python-like)
Convention	Yes (per plugin) + customizable	Yes (rigid, hard to customize)	Yes (BUILD files declare what's there)
Programmability	Full (it's a real language)	Limited (need to write Java plugin)	Limited (Starlark is sandboxed)
Incremental builds	Excellent (input/output tracking)	Coarse (per module)	Excellent (action graph + cache)
Build cache (local + remote)	Yes	No (per-module artifact only)	Yes (huge advantage at scale)
Hermetic builds	Best-effort (not enforced)	No	Strictly enforced
Android support	First-class (AGP)	Outdated, abandoned	Possible but rough
Used by	Android default; many JVM projects	Older Java/JVM projects	Google internal; Uber Android; some FAANG

4. Critical clarification — does Groovy/Kotlin DSL ship to users?

> **Common confusion:** "How does Android understand Groovy in build.gradle? Is the Groovy interpreter in DEX?"
> **Answer: No.** Build scripts are *build-time-only*. They run on your developer machine / CI server BEFORE the APK is created. They are NOT packaged in the APK and end users never see them.

What's actually inside an APK / AAB

FILE	WHAT
`classes.dex` (and classes2.dex, ...)	Your Kotlin/Java code compiled to **DEX bytecode** (Dalvik EXecutable format)
`AndroidManifest.xml`	Binary XML; describes activities, services, perms
`resources.arsc`	Compiled resource table
`res/`	Drawable XMLs, layout XMLs (binary), values
`assets/`	Raw asset files
`lib/<abi>/*.so`	Native libraries (per CPU ABI)
`META-INF/`	Signing manifest + signatures
`baseline-prof.txt` / `baseline.profm`	(Optional) Baseline Profile

That's it. **Zero Gradle, zero Groovy, zero build script content.**

The build flow at a glance

```
DEVELOPER MACHINE / CI SERVER (build time)
┌──────────────────────────────────────────────────────────────┐
│  *.kt source files  +  build.gradle.kts  +  resources/      │
│                            │                                 │
│              ┌─────────────┴──────────────┐                  │
│              ▼                            ▼                  │
│  1. Gradle reads build scripts (Groovy or Kotlin runtime)    │
│  2. Builds task DAG                                          │
│  3. Runs tasks: kotlinc → R8 → D8 (dex) → aapt2 → apksigner  │
└──────────────────────────────┬───────────────────────────────┘
                               ▼
                        ┌──────────────┐
                        │   APK / AAB  │   (no Groovy here!)
                        │  classes.dex │
                        │  resources   │
                        │  manifest    │
                        │  signatures  │
                        └──────┬───────┘
                               ▼
                       END USER DEVICE
                       ART runtime executes DEX bytecode
```

So when someone asks "how does Android understand Groovy", the answer is "it doesn't — and it doesn't need to. The Groovy/Kotlin in build scripts is interpreted/compiled at build time on the developer's machine; only the resulting compiled DEX bytecode is shipped to users, and ART (Android Runtime) executes that."

5. Build scripts: Groovy DSL (.gradle) vs Kotlin DSL (.gradle.kts)

	GROOVY (BUILD.GRADLE)	KOTLIN (BUILD.GRADLE.KTS)
Syntax	Dynamic, type-loose	Statically typed
IDE autocomplete	Limited	Excellent (it's Kotlin)
Refactoring support	Poor	Full Kotlin refactoring
Error visibility	Runtime	Compile-time (build-script compile)
Build script compile cost	Faster initial parse	Slower first compile, then cached
2026 default	Legacy projects	Recommended for new projects

Side-by-side

```
// Groovy DSL — build.gradle
android {
    compileSdk 34
    defaultConfig {
        applicationId "com.example.app"
        minSdk 24
    }
}
dependencies {
    implementation "androidx.core:core-ktx:1.13.1"
}
```

```
// Kotlin DSL — build.gradle.kts
android {
    compileSdk = 34
    defaultConfig {
        applicationId = "com.example.app"
        minSdk = 24
    }
}
dependencies {
    implementation("androidx.core:core-ktx:1.13.1")
}
```

Subtle differences: Kotlin requires explicit `=` for assignments and parentheses around method calls. The Kotlin DSL is the long-term direction; AGP and Gradle ship type-safe accessors that make Kotlin DSL ergonomic.

6. ★ Internals — the 3 phases of every Gradle build

This is the single most important Gradle concept. Every build runs through three phases in order:

```
┌──────────────────────────────────────────────────────────┐
│ PHASE 1 — INITIALIZATION                                 │
│  - Find settings.gradle.kts                              │
│  - Decide which projects participate (single vs multi-module)│
│  - Create a Project instance per module                  │
└──────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────┐
│ PHASE 2 — CONFIGURATION                                  │
│  - Execute every build.gradle.kts script (all modules)   │
│  - Each script registers tasks + plugins                 │
│  - Build the COMPLETE task graph (DAG)                   │
│  - NOTHING actually compiles or runs yet                 │
└──────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────┐
│ PHASE 3 — EXECUTION                                      │
│  - Determine which tasks the user asked for              │
│  - Walk the DAG, run only required tasks (in dep order)  │
│  - Skip tasks whose inputs/outputs are up-to-date        │
│  - Use build cache if available                          │
└──────────────────────────────────────────────────────────┘
```

Why this matters for performance

- **Configuration is expensive** — every script runs even if you only want to compile one module. Old Android projects took 30+ s in this phase alone.
- **Configuration cache** (Gradle 7.5+) serializes the configured task graph to disk; subsequent builds skip configuration entirely if nothing in the build scripts changed.
- **Avoid expensive work in configuration phase**: don't read files, query network, or call git from a build script's top level — it runs every build.
- **Use lazy task configuration**: `tasks.register("foo") { ... }` instead of `tasks.create("foo") { ... }` — register defers the body to execution phase.

The same thing in code

```
// In settings.gradle.kts — INITIALIZATION phase
include(":app", ":feature:home", ":core:network")

// In :app/build.gradle.kts — CONFIGURATION phase (runs once)
plugins {
    id("com.android.application")
    kotlin("android")
}

android {
    compileSdk = 34     // runs in configuration
}

tasks.register("printTime") {
    doLast {                // EXECUTION phase — runs only when task is invoked
        println("Built at ${java.time.Instant.now()}")
    }
}
```

7. The task graph (DAG) — how Gradle decides what runs

Every task has inputs (files, properties) and outputs (files). Tasks declare dependencies on other tasks via `dependsOn` or implicitly (via input being another task's output).

```
// Task A depends on Task B's output
tasks.register<Copy>("copyAsset") {
    from("src/main/assets")
    into(layout.buildDirectory.dir("copied"))
}

tasks.register("deploy") {
    dependsOn("copyAsset")         // explicit
    doLast { ... }
}
```

Up-to-date checks (the heart of Gradle's speed)

Each time a task runs, Gradle hashes its inputs and outputs. If both match the previous run, the task is **skipped** — saving full re-execution. This is why incremental builds in Gradle are blazing fast.

```
tasks.register("generate") {
    inputs.dir("schemas")              // inputs Gradle hashes
    inputs.property("version", "1.0")
    outputs.dir(layout.buildDirectory.dir("generated"))   // outputs Gradle tracks
    doLast { ... }
}
```

Build cache — across machines

The **build cache** stores task outputs keyed by input hashes. If teammate A built a task with the same inputs an hour ago, teammate B downloads the result instead of rebuilding. Configure with `--build-cache`; for remote cache use Gradle Enterprise or a self-hosted HTTP cache.

8. Anatomy of an Android `build.gradle.kts` — line by line

```
// app/build.gradle.kts

plugins {
    id("com.android.application")            // applies AGP — adds android{} extension and tasks like assemble
    kotlin("android")                           // Kotlin Android plugin
    id("com.google.devtools.ksp")             // KSP for annotation processing
    id("com.google.dagger.hilt.android")      // Hilt
}

android {
    namespace = "com.example.app"                // R class package + manifest tools
    compileSdk = 34                              // API level used at compile time

    defaultConfig {
        applicationId = "com.example.app"        // final package name on Play
        minSdk = 24                              // minimum supported API
        targetSdk = 34                           // API the app is designed for; affects runtime behaviors
        versionCode = 42                          // integer for Play to compare versions
        versionName = "1.4.2"                     // human-readable version
        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
    }

    buildTypes {
        getByName("release") {
            isMinifyEnabled = true                // run R8
            isShrinkResources = true             // strip unused resources
            proguardFiles(
                getDefaultProguardFile("proguard-android-optimize.txt"),
                "proguard-rules.pro"
            )
            signingConfig = signingConfigs.getByName("release")
        }
        getByName("debug") {
            applicationIdSuffix = ".debug"          // install side-by-side with release
            isDebuggable = true
        }
    }

    flavorDimensions += "environment"
    productFlavors {
        create("qa")  { dimension = "environment"; buildConfigField("String", "BASE_URL", "\"https://qa.api.ex
        create("prod") { dimension = "environment"; buildConfigField("String", "BASE_URL", "\"https://api.exam
    }

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
    }
    kotlinOptions {
```

```
        jvmTarget = "17"
    }
    buildFeatures {
        compose = true                              // enable Compose
        buildConfig = true                          // generate BuildConfig.kt
    }
    composeOptions {
        kotlinCompilerExtensionVersion = libs.versions.compose.compiler.get()
    }
    packaging {
        resources.excludes += "/META-INF/{AL2.0,LGPL2.1}"    // avoid META-INF conflicts
    }
}

dependencies {
    implementation(libs.androidx.core.ktx)               // from version catalog
    implementation(platform(libs.androidx.compose.bom))  // BOM = aligned versions
    implementation(libs.androidx.compose.ui)
    ksp(libs.hilt.compiler)
    testImplementation(libs.junit.jupiter)
    androidTestImplementation(libs.androidx.compose.test)
}
```

Every line above is doing a specific configuration call on the AGP / Kotlin extension objects. None of it executes the build — it sets up the configuration, then Gradle generates tasks based on what it sees.

9. `settings.gradle.kts` & root project

```
// settings.gradle.kts — runs in INITIALIZATION phase

pluginManagement {
    repositories {
        google()
        mavenCentral()
        gradlePluginPortal()
    }
}

dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)   // enforce: only use repos defined here
    repositories {
        google()
        mavenCentral()
    }
    versionCatalogs {
        create("libs") {
            from(files("gradle/libs.versions.toml"))
        }
    }
}

rootProject.name = "my-app"

include(
    ":app",
    ":core:network",
    ":core:database",
    ":feature:home",
    ":feature:profile"
)

// Optional: included builds for shared convention plugins
includeBuild("build-logic")
```

Key things `settings.gradle.kts` does:

- **pluginManagement**: where Gradle looks for plugin declarations
- **dependencyResolutionManagement**: where modules look for dependencies; centralizes repo declarations
- **versionCatalogs**: registers the libs.versions.toml file (next section)
- **include(...)**: declares which subprojects exist
- **includeBuild**: composite-build mechanism for sharing build logic across projects

10. Version Catalog (`libs.versions.toml`) — the modern way

One TOML file with all your versions, libraries, plugins. Type-safe accessors generated automatically. Replaces the old "ext.versions" Map pattern from pre-2022.

```
# gradle/libs.versions.toml

[versions]
kotlin           = "2.0.0"
agp              = "8.5.0"
compose-bom      = "2024.06.00"
compose-compiler = "1.5.14"
hilt             = "2.51.1"
ksp              = "2.0.0-1.0.21"

[libraries]
androidx-core-ktx           = { module = "androidx.core:core-ktx", version = "1.13.1" }
androidx-compose-bom        = { module = "androidx.compose:compose-bom", version.ref = "compose-bom" }
androidx-compose-ui         = { module = "androidx.compose.ui:ui" }
hilt-android                = { module = "com.google.dagger:hilt-android", version.ref = "hilt" }
hilt-compiler               = { module = "com.google.dagger:hilt-compiler", version.ref = "hilt" }

[plugins]
android-application = { id = "com.android.application", version.ref = "agp" }
android-library     = { id = "com.android.library", version.ref = "agp" }
kotlin-android      = { id = "org.jetbrains.kotlin.android", version.ref = "kotlin" }
hilt                = { id = "com.google.dagger.hilt.android", version.ref = "hilt" }
ksp                 = { id = "com.google.devtools.ksp", version.ref = "ksp" }

[bundles]
compose = ["androidx-compose-ui", "androidx-compose-material3", "androidx-compose-tooling"]
```

In your build files:

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.android)
}

dependencies {
    implementation(libs.androidx.core.ktx)
    implementation(platform(libs.androidx.compose.bom))
    implementation(libs.bundles.compose)
    ksp(libs.hilt.compiler)
}
```

Type-safe — IDE autocomplete works; renaming a version in the TOML auto-refactors everywhere it's used.

11. Build types vs flavors vs variants — the holy trinity

CONCEPT	WHAT IT IS	EXAMPLES
Build type	*How* to build it (signing, minification, debuggability)	debug, release, staging
Product flavor	*What* to build (which features, which env)	free vs paid; qa vs prod; tablet vs phone
Variant	The cross-product: `flavor + buildType`	`qaDebug`, `qaRelease`, `prodDebug`, `prodRelease`
Flavor dimension	An axis along which flavors group	"environment" (qa/prod), "tier" (free/paid) — 2 dimensions × 2 each = 4 flavors → 4 × 2 build types = 8 variants

```
android {
    buildTypes {
        debug   { applicationIdSuffix = ".debug"; isDebuggable = true }
        release { isMinifyEnabled = true }
    }

    flavorDimensions += listOf("environment", "tier")
    productFlavors {
        create("qa")   { dimension = "environment"; applicationIdSuffix = ".qa" }
        create("prod") { dimension = "environment" }
        create("free") { dimension = "tier"; applicationIdSuffix = ".free" }
        create("paid") { dimension = "tier" }
    }
}
// Variants generated: qaFreeDebug, qaFreeRelease, qaPaidDebug, qaPaidRelease,
//                     prodFreeDebug, prodFreeRelease, prodPaidDebug, prodPaidRelease
```

12. Source sets — what `qaImpl`, `prodImpl`, `debug` folders actually do

For every variant, AGP defines a hierarchy of source sets that get merged. **Files in flavor- or build-type-specific folders override or supplement `main` for that variant.**

```
project/app/src/
├── main/                  ← always included
│   ├── java/...            (or kotlin/)
│   ├── res/...
│   └── AndroidManifest.xml
│
├── debug/                 ← included in *Debug* variants only
│   ├── java/.../FakeApi.kt   (overrides main/.../FakeApi.kt if present)
│   └── res/values/strings.xml (merged with main; debug wins on conflict)
│
├── release/               ← included in *Release* variants only
│
├── qa/                    ← included in qa* variants only
│   └── java/.../Config.kt    (qa BASE_URL etc)
│
├── prod/                  ← included in prod* variants only
│
├── qaDebug/               ← qa AND debug — most specific
│
└── prodRelease/           ← prod AND release
```

What gets merged at compile time, in order

1. `main`
2. buildType folder (e.g., `debug` or `release`)
3. each flavor's folder (e.g., `qa`)
4. flavor + buildType combination (e.g., `qaDebug`)
5. multi-flavor combinations

Higher specificity wins for **code** (the variant-specific class replaces `main`'s) and is **merged** for **resources/manifest**.

What "qaImpl" really means

The naming convention "qaImpl" you might see in dependencies refers to the **configuration**, not a folder:

```
dependencies {
    implementation(libs.androidx.core.ktx)             // always included
    qaImplementation(libs.flipper)                     // only in qa* variants
    debugImplementation(libs.leakcanary)               // only in *Debug variants
    qaDebugImplementation(libs.chuckerteam.chucker)    // only in qaDebug variant
}
```

For each variant, AGP creates configurations like `qaImplementation`, `debugImplementation`, `qaDebugImplementation`, etc. Adding a dependency to `debugImplementation` means it's only on the classpath for debug variants. *This is how LeakCanary stays out of your release APK.*

13. Dependency configurations — `implementation` vs `api` vs `compileOnly` vs more

CONFIGURATION	MEANING	EXAMPLE
`implementation`	Default. Used at compile + runtime. Hidden from consumers.	Most things
`api`	Like implementation but EXPOSED — types are part of your public API; consumers see them transitively.	Common-types module exposing models
`compileOnly`	Available at compile, NOT at runtime. Caller must provide at runtime.	Annotation libs (jakarta.annotation), ServiceLoader interfaces
`runtimeOnly`	NOT at compile, only at runtime.	JDBC drivers; logging implementations
`ksp` / `kapt`	Annotation processors (compile-time only)	Hilt, Room, Moshi-codegen
`testImplementation`	Only for unit tests (src/test/)	JUnit, MockK, Turbine
`androidTestImplementation`	Only for instrumented tests (src/androidTest/)	Espresso, Compose Test
`debugImplementation`	Only in *Debug variants	LeakCanary, Chucker
`releaseRuntimeOnly`	Only in *Release variants, runtime only	Logging redactor

The api/implementation split — why it matters for build speed

```
// In :core:network
dependencies {
    api(libs.retrofit)                  // consumers can import retrofit.* directly
    implementation(libs.okhttp)         // consumers cannot see okhttp; must declare separately
}

// In :feature:home (depends on :core:network)
dependencies {
    implementation(project(":core:network"))
    // can use Retrofit (api), cannot use OkHttp (impl)
}
```

Why prefer `implementation`? Changes to OkHttp recompile only the network module, not every consumer module. Switching from `api` to `implementation` on a heavily-used module can cut incremental build time by 20-40%.

Conflict resolution — when two deps want different versions

By default, Gradle picks the highest version. Override with:

```
configurations.all {
    resolutionStrategy {
        force("com.squareup.okhttp3:okhttp:4.12.0")         // pin a version
        eachDependency {
            if (requested.group == "com.squareup.okhttp3") {
                useVersion("4.12.0")
            }
        }
    }
}
```

For diagnosing version conflicts: `./gradlew :app:dependencies --configuration releaseRuntimeClasspath`

14. Custom tasks — writing your own

Simple ad-hoc task

```
tasks.register("hello") {
    group = "misc"                        // shows up under "misc" in ./gradlew tasks
    description = "Prints hello"
    doLast {
        println("Hello from Gradle")
    }
}
// Run: ./gradlew hello
```

Typed task (custom Task class)

```kotlin
abstract class GenerateBuildInfoTask : DefaultTask() {
    @get:Input
    abstract val versionName: Property<String>

    @get:OutputFile
    abstract val outputFile: RegularFileProperty

    @TaskAction
    fun generate() {
        val file = outputFile.get().asFile
        file.writeText("object BuildInfo { const val VERSION = \"${versionName.get()}\" }")
    }
}

tasks.register<GenerateBuildInfoTask>("genBuildInfo") {
    versionName.set("1.4.2")
    outputFile.set(layout.buildDirectory.file("generated/BuildInfo.kt"))
}

// Wire as a source for compilation
android {
    sourceSets["main"].kotlin.srcDir(layout.buildDirectory.dir("generated"))
}
```

Typed tasks unlock incremental builds

- `@Input` — properties whose value affects output. Gradle hashes them.
- `@OutputFile` / `@OutputDirectory` — Gradle tracks for up-to-date check.
- `@InputFiles` — file inputs.
- `@TaskAction` — the work to do.
- If inputs + outputs unchanged → task SKIPPED. Cacheable across machines if marked `@CacheableTask`.

Common Gradle DSL keywords

KEYWORD	WHAT IT DOES
`tasks.register("name")`	Lazily register a task — body deferred to needed
`tasks.create("name")`	Eagerly create — runs in configuration phase. Avoid.
`tasks.named("name")`	Get a registered task lazily
`doFirst { }`	Add behavior at the start of execution
`doLast { }`	Add behavior at the end
`dependsOn(...)`	Explicit task dependency
`finalizedBy(...)`	Always run X after Y, even if Y fails
`mustRunAfter(...)`	If both run, X must come after Y (no implicit ordering)
`onlyIf { }`	Skip task unless condition true
`project.exec { }`	Run an external process (avoid in config phase!)
`providers.exec { }`	Configuration-cache-friendly version

15. Plugins, buildSrc, convention plugins, included builds

You apply plugins — but how do you write your own to share logic across modules? Three options, evolving in sophistication:

1. Inline ext blocks (legacy, painful)

```
// Old way: copy-paste android{} block into 10 modules
```

2. `buildSrc/` — Gradle's special folder

Anything in `buildSrc/` is compiled once and made available to all build scripts. Place shared Kotlin code there, then call from any module.

```
buildSrc/
├── build.gradle.kts
└── src/main/kotlin/
    └── Versions.kt
        object Versions { const val kotlin = "2.0.0"; ... }
```

Downside: changing buildSrc invalidates ALL build script caches → slow CI.

3. ★ Convention plugins (the modern way)

Write small plugin classes in a separate Gradle build (`build-logic/`) that you `includeBuild`. Each plugin applies a "convention" — a standard configuration for a category of module (Android lib, Compose feature, JVM lib). Modules just apply the convention plugin and get everything.

```kotlin
// build-logic/convention/src/main/kotlin/AndroidLibraryConventionPlugin.kt
class AndroidLibraryConventionPlugin : Plugin<Project> {
    override fun apply(target: Project) = with(target) {
        plugins.apply("com.android.library")
        plugins.apply("org.jetbrains.kotlin.android")

        extensions.configure<LibraryExtension> {
            compileSdk = 34
            defaultConfig.minSdk = 24
            compileOptions {
                sourceCompatibility = JavaVersion.VERSION_17
                targetCompatibility = JavaVersion.VERSION_17
            }
        }

        dependencies {
            add("implementation", "androidx.core:core-ktx:1.13.1")
        }
    }
}

// settings.gradle.kts of build-logic — register the plugin id
gradlePlugin {
    plugins {
        register("androidLibrary") {
            id = "myapp.android.library"
            implementationClass = "AndroidLibraryConventionPlugin"
        }
    }
}

// In any module: just one line!
plugins {
    id("myapp.android.library")
}
```

This is how Now in Android (Google's reference app) and most large enterprise codebases scale build logic. Updating Kotlin version = change one file in `build-logic/`; all 50 modules pick it up.

4. Included builds (composite builds)

`includeBuild("path/to/other-project")` in `settings.gradle.kts` — treats another Gradle build as a dependency. Useful for working on a library + app together without publishing.

16. Gradle wrapper, daemon, and version management

The wrapper — why every project ships `./gradlew`

The `gradle/wrapper/gradle-wrapper.properties` file pins the exact Gradle version. `./gradlew` downloads that version on first run. Result: every developer + CI runs the same Gradle. Don't edit; let Gradle update via `./gradlew wrapper --gradle-version 8.7`.

```
# gradle/wrapper/gradle-wrapper.properties
distributionBase=GRADLE_USER_HOME
distributionPath=wrapper/dists
distributionUrl=https\://services.gradle.org/distributions/gradle-8.7-bin.zip
zipStoreBase=GRADLE_USER_HOME
zipStorePath=wrapper/dists
networkTimeout=10000
distributionSha256Sum=...
```

The Gradle daemon — why builds are fast (after the first)

Gradle starts a long-lived JVM daemon process that stays warm between builds. Subsequent builds reuse it — JIT-compiled, classes loaded, build script compiled. Configuration cache + daemon = sub-second incremental builds for many projects.

SETTING	EFFECT
`org.gradle.daemon=true` (default)	Keep daemon alive across builds
`org.gradle.parallel=true`	Run independent tasks in parallel across modules
`org.gradle.caching=true`	Enable build cache (re-use task outputs)
`org.gradle.configuration-cache=true`	Skip configuration phase if scripts unchanged
`org.gradle.jvmargs=-Xmx4g -XX:MaxMetaspaceSize=1g`	Heap + metaspace for the daemon
`org.gradle.workers.max=8`	Cap parallel workers (default: cpuCount)

Set in `gradle.properties`.

Gradle version vs AGP version vs Kotlin version

COMPONENT	WHERE PINNED
Gradle	gradle/wrapper/gradle-wrapper.properties (distributionUrl)
Android Gradle Plugin (AGP)	plugins block in :app/build.gradle.kts (or version catalog)
Kotlin	plugins block (Kotlin Android plugin)
Compose Compiler	composeOptions.kotlinCompilerExtensionVersion (must match Kotlin version compatibility)
KSP	plugin id (must match Kotlin version, e.g., 2.0.0-1.0.21)

The compatibility matrix is real. Wrong AGP for Gradle = build fails immediately. Wrong Kotlin for KSP = obscure error. Always check the AGP release notes before bumping.

17. ★ Configuration cache, build cache, parallelism — the perf trio

Configuration cache (Gradle 7.5+)

Serializes the configured task graph to disk after the first run. Subsequent builds load it directly — skipping the entire configuration phase. Result: 10s configuration → 100ms.

Enable: `org.gradle.configuration-cache=true`.

Restrictions for compatibility:

- Tasks must not capture `Project` at execution time — use `providers` and `layout`
- No `System.getenv()` at config time — use `providers.environmentVariable("X")`
- External processes during config: use `providers.exec { }` not `project.exec { }`

Run `./gradlew <task> --configuration-cache` to see incompatibilities. Most third-party plugins are now compatible.

Build cache

Stores task *outputs* keyed by input hashes. Two layers:

- **Local** — `~/.gradle/caches/build-cache-1/`. Across projects on your machine.
- **Remote** — Gradle Enterprise / DIY HTTP cache. Across machines (CI shares with devs).

For task A, Gradle hashes (inputs + classpath + JVM args + Gradle version + plugin versions). If a cache hit, downloads outputs and skips execution. Massive win on CI.

Parallelism

Two levels:

- **Project parallelism** (`org.gradle.parallel=true`): run tasks of independent modules in parallel
- **Worker API**: parallel work within a single task (e.g., dexing in parallel chunks; AGP uses this internally)

How they combine

```
First build:    config (slow) → execution (slow)         = ~30 s
Cached config:  cached config (fast) → execution (slow)   = ~15 s
+ build cache:  cached config (fast) → cached exec (fast) = ~3 s
+ parallel:     even faster on multi-module                = ~1 s
```

18. KSP vs kapt — annotation processing

	KSP	KAPT
How it works	Native Kotlin Symbol Processing (Kotlin compiler API)	Generates Java stubs from Kotlin, runs Java annotation processors
Speed	~2-3× faster than kapt	Slow (extra stub-generation step)
Incremental support	Excellent	OK
Library support	Hilt, Room, Moshi-codegen, Glide (and more) — most major libs	Anything not yet on KSP
2026 default	Yes	Only if a library hasn't migrated

```
// Switch from kapt to ksp:
plugins {
    id("com.google.devtools.ksp") version "2.0.0-1.0.21"
    // kotlin("kapt") — remove
}
dependencies {
    ksp(libs.hilt.compiler)         // was kapt(libs.hilt.compiler)
    ksp(libs.room.compiler)
}
```

19. AGP (Android Gradle Plugin) — what it adds

AGP teaches Gradle how to build Android. Without it, Gradle doesn't know what an APK is.

WHAT AGP ADDS	EXAMPLES
The `android { }` extension	Where you configure compileSdk, build types, flavors
Android-aware tasks	assembleDebug, bundleRelease, lintDebug, mergeReleaseResources, processReleaseResources, dexBuilder, etc.
Source set merging	main + variant-specific folders
R8 integration	Shrink/optimize/obfuscate at release
Manifest merger	Combines library manifests with your app's
Resource processing (AAPT2)	Compiles XML resources to binary
D8 dexing	Converts JVM bytecode → DEX
Variant API	Programmatic access to build variants — for custom build logic

The Variant API — power users only

```
androidComponents {
    onVariants { variant ->
        // Programmatically modify a variant's setup
        if (variant.name == "prodRelease") {
            variant.outputs.forEach {
                if (it is VariantOutputImpl) {
                    it.outputFileName.set("my-app-${variant.versionName.get()}.apk")
                }
            }
        }
    }
}
```

20. What actually happens when you run `./gradlew assembleRelease`

```
1. INITIALIZATION
   - Read settings.gradle.kts
   - Discover all included projects
   - Create Project instances

2. CONFIGURATION (or load from configuration cache)
   - Run every build.gradle.kts
   - Apply plugins (AGP, Kotlin, Hilt, KSP, …)
   - Build the task DAG

3. EXECUTION — for each task in topological order:
   ┌─ preBuild
   ├─ generateReleaseSources           (BuildConfig, R, Hilt-generated, KSP-generated)
   ├─ compileReleaseKotlin             (kotlinc → JVM bytecode in build/intermediates)
   ├─ kspReleaseKotlin                 (annotation processors run)
   ├─ compileReleaseJavaWithJavac      (any Java sources)
   ├─ mergeReleaseResources            (res/ folders merged across variant)
   ├─ processReleaseResources          (AAPT2 — XML → binary)
   ├─ minifyReleaseWithR8              (shrink + optimize + obfuscate JVM bytecode)
   ├─ dexBuilderRelease (D8)           (JVM bytecode → DEX)
   ├─ mergeDexRelease                  (combine into classes.dex, classes2.dex, …)
   ├─ packageRelease                   (zip everything into APK structure)
   ├─ apkSignerRelease                 (sign with release keystore)
   └─ assembleRelease                  (final outputs in build/outputs/apk/release/)
```

For an AAB (App Bundle), replace the last steps with `bundleRelease` which produces an .aab file Play uses to generate per-device split APKs.

21. Signing config & release variants

```
android {
    signingConfigs {
        create("release") {
            // Don't put real secrets in the build file!
            storeFile     = file(System.getenv("KEYSTORE_PATH") ?: "missing.keystore")
            storePassword = System.getenv("KEYSTORE_PWD")
            keyAlias      = System.getenv("KEY_ALIAS")
            keyPassword   = System.getenv("KEY_PWD")
        }
    }
    buildTypes {
        getByName("release") {
            signingConfig = signingConfigs.getByName("release")
        }
    }
}
```

Modern flow: **Play App Signing** — Google holds the actual signing key; you upload signed with your "upload key." Lower compromise blast radius. Mandatory for new apps on Play.

22. Multi-module patterns

Recommended structure

```
:app                          # thin glue: NavHost, Application, theme
:build-logic                  # includeBuild, hosts convention plugins
:core
  :core:common                # utilities, KMP-ready
  :core:design-system         # Composables, theme
  :core:network               # OkHttp, Retrofit, common interceptors
  :core:database              # Room DB, DAOs
  :core:domain                # shared models, interfaces
  :core:test-utils            # shared fakes, test rules
:feature
  :feature:home               # HomeScreen + ViewModel + UseCases + Repo
  :feature:profile
  :feature:checkout
```

Dependency rules

- **:app** depends on every feature module
- **:feature:*** depends on **:core:*** only — never on other features
- **:core:*** are leaves — no upward dependencies
- Cross-feature navigation goes through a navigator interface in `:core:domain`, implemented in `:app`

Type-safe project accessors (Gradle 7+)

```
// settings.gradle.kts
enableFeaturePreview("TYPESAFE_PROJECT_ACCESSORS")

// In any build.gradle.kts
dependencies {
    implementation(projects.core.network)         // type-safe!
    implementation(projects.core.database)
}
// vs the old way: implementation(project(":core:network"))
```

23. Build performance — the checklist

1. **Use the daemon** + parallel + caching + configuration cache. The default `gradle.properties` for any project:

```
org.gradle.daemon=true
org.gradle.parallel=true
org.gradle.caching=true
org.gradle.configuration-cache=true
org.gradle.jvmargs=-Xmx4g -XX:MaxMetaspaceSize=1g
android.useAndroidX=true
android.nonTransitiveRClass=true
kotlin.incremental=true
```

2. **Switch to KSP** from kapt — 2-3× faster annotation processing
3. **Use `implementation` instead of `api`** — limits transitive recompilation
4. **Use convention plugins via `build-logic`** — avoids recompiling buildSrc on every change
5. **Avoid expensive work in configuration phase** — no I/O, no exec, no git calls at top level
6. **Use lazy task registration** (`tasks.register` over `tasks.create`)
7. **Profile with `--scan`** — `./gradlew assembleDebug --scan` uploads a build scan to Gradle Enterprise; tells you exactly what was slow
8. **Profile with `--profile`** — generates an HTML report locally
9. **Module size matters** — sweet spot 10-30 modules; over-modularization slows configuration
10. **Enable remote build cache on CI** — devs share results with CI; ~30-60% faster

24. Common Gradle problems & fixes

PROBLEM	CAUSE	FIX
"Daemon disappeared unexpectedly"	Out of memory or killed by OS	Increase `org.gradle.jvmargs=-Xmx4g`
"Could not resolve all artifacts"	Dependency repos misconfigured	Check repositories in settings.gradle.kts; ensure offline mode off
"Duplicate class found in modules"	Two libraries shipping the same classes	Add `configurations.all { exclude(group = "x", module = "y") }`
"Plugin not found"	Missing repository in pluginManagement	Add gradlePluginPortal() to pluginManagement.repositories
"R8 stripped my class"	Reflection access without -keep rule	Add appropriate -keep rule in proguard-rules.pro
"Configuration cache problem: ..."	Plugin/code captures Project at execution time	Refactor to use providers/layout APIs; or disable for that task
"Gradle version not compatible with AGP"	Wrong wrapper version for your AGP	Bump wrapper: `./gradlew wrapper --gradle-version 8.7`
"Dex limit exceeded (65536)"	Too many methods for single DEX	Enable multidex or use R8 minification
"OutOfMemoryError: Metaspace"	Too many classes loaded in build process	Increase `-XX:MaxMetaspaceSize=1g`
"Task ':compileKotlin' is not up-to-date because input has changed"	Sometimes spurious — caches got confused	`./gradlew clean`; if recurring, file an issue
Slow first build after gradle.properties change	Daemon picks up new JVM args = restart	Expected; subsequent builds are fast

25. Common mistakes & anti-patterns

DON'T

- Hardcode versions in module build files (use Version Catalog)
- Use `api` for everything (kills incremental builds)
- Run `System.getenv()` at config time (breaks config cache)
- Read files in build script top-level (also breaks config cache)
- Use `tasks.create` instead of `tasks.register`
- Put secrets in build files (use env vars or signing config files)
- Modify task graph in afterEvaluate {} unless absolutely needed
- Mix Groovy and Kotlin DSL in the same project (pick one)
- Use `compile` configuration (deprecated long ago)
- Inflate buildSrc with logic that should be a plugin
- Forget to commit `gradle/` + wrapper jar (others can't build)
- Use `android.disableAutomaticComponentCreation = true` blindly

DO

- Use Version Catalog for all versions
- Prefer `implementation` over `api`
- Use `tasks.register` for laziness
- Use convention plugins in `build-logic/` for shared config
- Enable configuration cache + build cache + parallel
- Run `--scan` when builds feel slow
- Use type-safe project accessors (`projects.core.network`)
- Switch to KSP from kapt where possible
- Use `libs.versions.toml` bundles for grouped deps (`libs.bundles.compose`)
- Pin Gradle wrapper version in git
- Profile periodically — build times grow silently
- Read AGP release notes before bumping versions

26. Interview Q&A — FAANG-level

Q: "Walk me through what happens when I run `./gradlew assembleRelease`."

Three phases. **Initialization**: read settings.gradle.kts, discover modules, create Project instances. **Configuration**: run every module's build.gradle.kts, apply plugins, build task DAG (or load from configuration cache). **Execution**: walk the DAG topologically, run tasks (preBuild → kotlin compile → KSP processing → R8 → D8 dex → resource merge → AAPT2 → package → sign → assemble), skipping anything up-to-date or in build cache. Output: APK in build/outputs/apk/release.

Q: "Difference between configuration phase and execution phase — and why it matters."

Configuration runs every script even for unrelated tasks. Execution runs only what's needed. Anything heavy in configuration costs every single build — file I/O, network calls, git commands. Use `tasks.register` (lazy) over `tasks.create` (eager) so task body runs in execution. Configuration cache makes this gap less painful by skipping configuration entirely on subsequent builds.

Q: "How does Gradle know if a task can be skipped?"

Up-to-date checks via input/output tracking. Each typed task declares `@Input`, `@InputFiles`, `@OutputFile`, `@OutputDirectory`. Gradle hashes them; if hash matches the previous run, task is SKIPPED. With `@CacheableTask`, also stored in build cache for cross-machine sharing.

Q: "How does build cache differ from configuration cache?"

Configuration cache stores the configured task graph (skips configuration phase). Build cache stores task *outputs* (skips execution of cached tasks). Both can apply to the same build: configuration cache hits → fast configuration; build cache hits → skipped task execution.

Q: "Why prefer `implementation` over `api`?"

`implementation` hides the dep from consumers — they can't import its types. Changing the dep doesn't recompile downstream modules. `api` exposes the dep — consumers see and can use it, but every change there recompiles everyone. Switching from `api` to `implementation` on a heavily-used internal module can cut build time 20-40%.

Q: "Build types vs flavors vs variants?"

Build types = HOW (debug/release/staging — minification, signing, debuggability). Flavors = WHAT (qa/prod, free/paid — features, environments). Variants = the cross-product (qaDebug, prodRelease, etc.). Each variant has its own merged source set and configurations like `qaDebugImplementation`.

Q: "What does `debugImplementation(libs.leakcanary)` actually do?"

AGP creates per-variant configurations. `debugImplementation` means the dependency is on the classpath only for *Debug variants — never in release builds. That's how LeakCanary stays out of production APKs.

Q: "Where is the Groovy/Kotlin from build.gradle when the user has the app?"

Nowhere. Build scripts are build-time-only — they run on developer/CI machines. The output APK contains compiled DEX bytecode (your Kotlin app code), resources, manifest, and signing — no Gradle, no Groovy. ART executes the DEX bytecode at runtime.

Q: "What's a convention plugin and why use one?"

A small Gradle plugin that bundles standard module configuration. You write it once in a `build-logic/` included build, then apply it to any module via `plugins { id("myapp.android.feature") }`. Updating Kotlin version or compileSdk = change one file, all 50 modules pick it up. The modern replacement for buildSrc + ext blocks.

Q: "Why is the Gradle daemon important for build speed?"

Gradle is a JVM tool. Cold-start JVM overhead is several hundred ms. The daemon is a long-lived JVM process that stays warm: classes loaded, JIT warm, build script compiled. Subsequent builds reuse it. Combined with configuration cache + build cache, incremental builds drop from 10+ s to sub-second.

Q: "How does Gradle resolve dependency version conflicts?"

Default: pick the highest version among all transitives. Override via `resolutionStrategy.force(...)` or `resolutionStrategy.eachDependency { useVersion(...) }`. Inspect with `./gradlew :app:dependencies --configuration releaseRuntimeClasspath` — shows the full tree with which version won and why.

Q: "What's KSP and why use it over kapt?"

KSP = Kotlin Symbol Processing. Native Kotlin compiler API for annotation processing. kapt generates Java stubs from Kotlin then runs Java APs — slow extra step. KSP processes Kotlin directly. ~2-3× faster, more accurate type info, better incremental support. Hilt, Room, Moshi-codegen all support KSP. Switch every project that can.

Q: "How do you debug a slow Gradle build?"

`./gradlew <task> --scan` uploads a build scan to Gradle Enterprise — shows configuration vs execution time, slow tasks, dependency resolution time, etc. Locally: `--profile` generates an HTML report. Enable `org.gradle.configuration-cache=true` and check whether config phase is the bottleneck.

Q: "What is the AGP and what does it add to Gradle?"

Android Gradle Plugin teaches Gradle how to build Android. Adds: the `android { }` extension, all the Android-aware tasks (assembleRelease, mergeResources, R8, dexBuilder, etc.), source set merging logic for variants, manifest merger, AAPT2 integration, R8 integration. Without AGP, Gradle has no idea what an APK is.

Q: "What's the difference between buildSrc and an included build for build-logic?"

`buildSrc` is a special Gradle folder that's compiled and added to all build script classpaths. Simple, but: changing anything in `buildSrc` invalidates all build script caches → painful on CI. `includeBuild("build-logic")` treats build-logic as a separate Gradle build with its own classpath; only changes to the specific plugin you're touching trigger recompilation. Modern recommendation: `build-logic` with included build.

27. One-page recall card

The mental model

Gradle = task DAG executor. Three phases: Init → Configuration → Execution. Build scripts are build-time only — the APK contains compiled DEX, not Groovy/Kotlin source. Configuration phase is expensive; configuration cache skips it.

Build types vs flavors vs variants

- Build type = HOW (debug/release: minify, sign, debuggable)
- Flavor = WHAT (qa/prod, free/paid)
- Variant = flavor × buildType (qaDebug, prodRelease)
- Source sets merged: main + buildType + flavor + variant
- Per-variant configs: `debugImplementation`, `qaImplementation`, etc.

Dependency configurations

- `implementation` — default; hidden from consumers
- `api` — exposed; consumers see types
- `compileOnly` — compile only, not packaged
- `runtimeOnly` — packaged, not at compile
- `ksp` / `kapt` — annotation processors
- `testImplementation` / `androidTestImplementation` — test scopes

Performance trio

- Daemon (`org.gradle.daemon=true`)
- Parallel (`org.gradle.parallel=true`)
- Build cache (`org.gradle.caching=true`)
- Configuration cache (`org.gradle.configuration-cache=true`)
- KSP over kapt
- implementation over api
- Convention plugins via build-logic included build
- Version Catalog for shared versions

What ships in the APK

- classes.dex (compiled Kotlin/Java)
- AndroidManifest.xml (binary)
- resources.arsc + res/
- assets/
- lib/<abi>/*.so
- META-INF/ (signing)
- baseline-prof.txt (optional)
- NOT: Gradle, Groovy, build scripts

Avoid

- `tasks.create` (eager) — use `tasks.register`
- I/O / git / env in config phase top-level
- Hardcoded versions per module
- `api` when `implementation` works
- kapt when KSP works
- Mixing Groovy + Kotlin DSL
- Secrets in build files
- Bloated buildSrc (use convention plugins)

Read top to bottom for understanding · skim §6 (3 phases), §17 (perf trio), §26 (Q&A), §27 (recall) before any build-system interview. Pair with the rest of the cheat sheets in this repo.